Sahad Rafiuzzaman

CS 580: Introduction to Artificial Intelligence

Dr. Lusi Li

February 11, 2025

Analysis Report for Homework Assignment 1

The 15-Puzzle Problem Solver is built on the original eight puzzle codes provided by the AIMA GitHub library. It uses a graphical user interface (GUI) using Tkinter. The GUI application has two parts: the puzzle and button frames. The puzzle frame visually represents the puzzle and its initial state. The button frame covers three different buttons. The first button is the scramble button, randomly scrambling the puzzle for the user to experience a new challenge. The second button is the Check Algorithms button, which runs the algorithms and outputs the performance metrics in the console. The third button is the Solve button, which uses the best algorithm depending on the least time required to solve, memory consumption, and the least amount of nodes expanded. Figure 1 is a screenshot of the graphical application with the buttons.

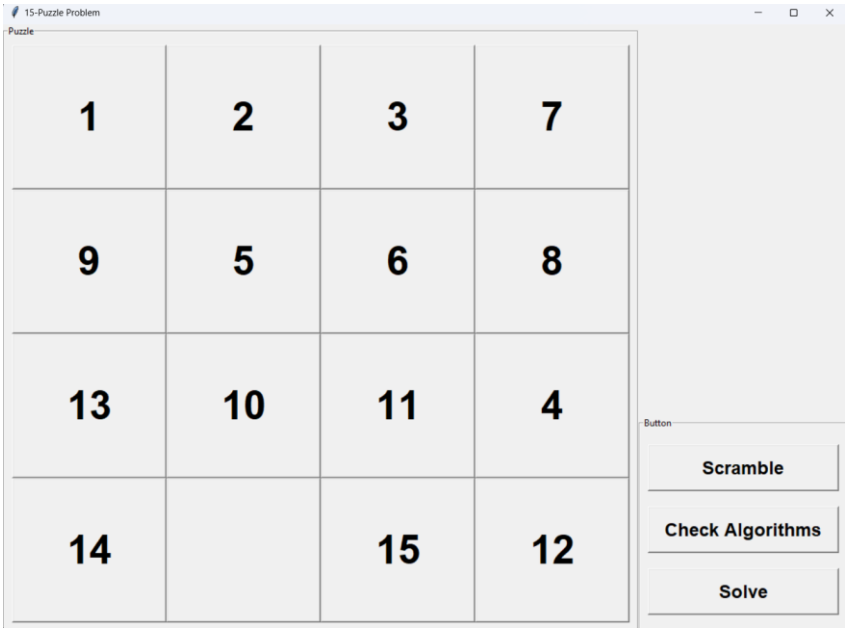

Figure 1: Graphical Application for the 15-Puzzle Problem Solver

The program compares three different search algorithms to solve the puzzle. The first algorithm is A* Search, an informed heuristic search algorithm. This algorithm solved the puzzle with the least number of expanded nodes, runtime, and memory usage. The Breadth-First Graph Search was the second type of algorithm used in this program. This algorithm took the second most time to find the solution to the puzzle. It also had a significantly higher quantity of nodes expanded, runtime, and memory usage than the IHS counterpart. The last algorithm used in this program is a variant of Depth First Search (DFS) named Iterative Deepening Search (IDDFS). The DFS options in the provided search.py ran quite unreliable. The moves for the DFS either got stuck in an infinite loop or output completely unreasonable moves for the solution. However, this variant, IDDFS,

proved significantly more reliable and could output the correct solution. During the runs of IDDFS, I observed that it was a much slower algorithm to find the solution to the puzzle. Figure 2 shows the output of all three algorithms alongside their respective performance metrics. The performance metrics are measured using tracemalloc and time libraries.

```
Searching Strategy: Informed Heuristic Search (IHS)
Moves: DLURDLDDLURRD
Nodes Expanded: 61
Time Taken: 3.0 ms
Memory Used: 49.3 KB
Solution Exists: True

Searching Strategy: Breadth-First Graph Search (BFS)
Moves: DLURDLDDLURRD
Nodes Expanded: 9778
Time Taken: 115.2 ms
Memory Used: 6761.7 KB
Solution Exists: True

Searching Strategy: Iterative Deepening Search (IDDFS)
Moves: DLURDLDDLURRD
Nodes Expanded: 943258
Time Taken: 7509.4 ms
Memory Used: 11.7 KB
Solution Exists: True
```

Figure 2: Console Output of Performance Metrics for Each Algorithm

The program selects the fastest algorithm; if two algorithms have the same speed, the lower memory usage is preferred. If the two algorithms have the same memory usage, the algorithm with the fewer expanded nodes is selected. However, in this assignment, I observed that the IHS algorithm was always the fastest and most efficient choice.

A minor downside of the program is that it runs each algorithm sequentially instead of in parallel. Running the algorithms in parallel could get the program to run more effectively and alleviate the temporary unresponsive state of the GUI. One problem that I faced during this assignment was trying to output the results on the graphical application in an output table. Figure 3 shows my original GUI for this program. My issue was that it would ultimately crash when it ran all the algorithms, even if the puzzle solutions were simple. If provided more time, I would have been able to debug my issues on the graphical application. Regardless, this assignment was an excellent opportunity for me to relearn Tkinter alongside new algorithms.

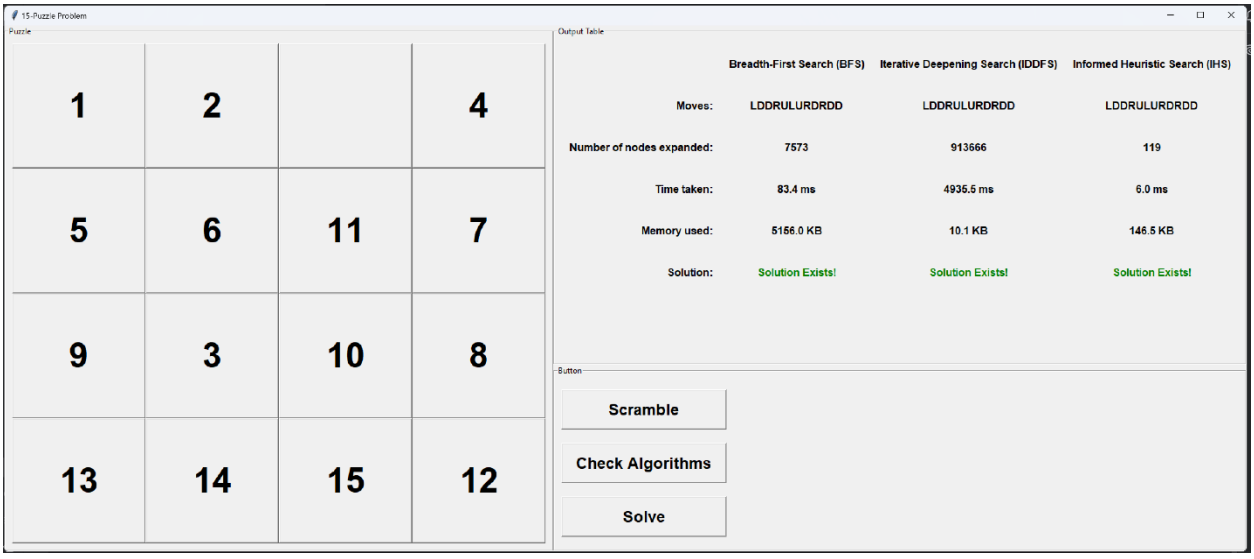

Figure 3: My original GUI for this 15-Puzzle Program Solver